

solid principles example with Below is an **interview-ready Payroll Management System example** for SOLID principles.

Below is an **interview-ready Payroll Management System example** for all five **SOLID principles** in C#. It is designed so you can explain both the definition and a realistic project example during a senior .NET interview.

SOLID Principles – Payroll Management System

SOLID stands for:

Principle	Meaning
S	Single Responsibility Principle
O	Open/Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

1. S — Single Responsibility Principle (SRP)

Definition: A class should have only **one reason to change**. In other words, one class should handle one responsibility.

Bad Design

Suppose one `PayrollService` calculates salary, saves payroll, generates payslips, and sends emails.

```
public class PayrollService
{
    public decimal CalculateSalary(Employee employee)
    {
        return employee.BasicSalary +
```

```

        employee.Allowance -
        employee.Deduction;
    }

    public void SavePayroll(Employee employee, decimal salary)
    {
        // Save to SQL Server
    }

    public void GeneratePayslip(Employee employee)
    {
        // Generate PDF
    }

    public void SendPayslipEmail(Employee employee)
    {
        // Send email
    }
}

```

This class has **four reasons to change**:

- Salary calculation changes
- Database logic changes
- Payslip format changes
- Email implementation changes

So it violates SRP.

Better Design

Separate each responsibility.

```

public class SalaryCalculator
{
    public decimal Calculate(Employee employee)
    {
        return employee.BasicSalary +
            employee.Allowance -
            employee.Deduction;
    }
}

```

```

public class PayrollRepository
{
    public void Save(Employee employee, decimal salary)
    {
        // Save to SQL Server
    }
}

```

```
{  
    // Database operation  
}  
}
```

```
public class PayslipGenerator  
{  
    public void Generate(Employee employee)  
    {  
        // Generate payslip  
    }  
}
```

```
public class EmailService  
{  
    public void SendPayslip(Employee employee)  
    {  
        // Send email  
    }  
}
```

Interview answer

In our payroll application, I wouldn't keep salary calculation, database operations, payslip generation, and email notification inside one service. I separate them into `SalaryCalculator`, `PayrollRepository`, `PayslipGenerator`, and `EmailService`. This follows SRP because each class has only one reason to change.

2. O — Open/Closed Principle (OCP)

Definition: Software entities should be **open for extension** but **closed for modification**.

Imagine we have different employee types:

- Permanent employee
- Contract employee
- Consultant

Bad Design

```

public decimal CalculateSalary(Employee employee)
{
    if (employee.Type == "Permanent")
    {
        return employee.BasicSalary + employee.HRA;
    }
    else if (employee.Type == "Contract")
    {
        return employee.HoursWorked * employee.HourlyRate;
    }
    else if (employee.Type == "Consultant")
    {
        return employee.DaysWorked * employee.DailyRate;
    }

    return 0;
}

```

Tomorrow, if the company introduces:

```

Intern
PartTime
Freelancer
Temporary

```



we have to keep modifying the same method.

That violates OCP.

Better Design

Create an abstraction.

```

public interface ISalaryCalculator
{
    decimal CalculateSalary();
}

```

Permanent employee:

```

public class PermanentEmployeeSalary : ISalaryCalculator
{
    public decimal BasicSalary { get; set; }
    public decimal HRA { get; set; }
}

```

```
    public decimal CalculateSalary()
    {
        return BasicSalary + HRA;
    }
}
```

Contract employee:

```
public class ContractEmployeeSalary : ISalaryCalculator
{
    public int HoursWorked { get; set; }
    public decimal HourlyRate { get; set; }

    public decimal CalculateSalary()
    {
        return HoursWorked * HourlyRate;
    }
}
```

Consultant:

```
public class ConsultantSalary : ISalaryCalculator
{
    public int DaysWorked { get; set; }
    public decimal DailyRate { get; set; }

    public decimal CalculateSalary()
    {
        return DaysWorked * DailyRate;
    }
}
```

Now add an intern:

```
public class InternSalary : ISalaryCalculator
{
    public decimal Stipend { get; set; }

    public decimal CalculateSalary()
    {
        return Stipend;
    }
}
```

We didn't modify existing salary calculators.

Interview answer

In payroll, employee categories can have different salary rules. Instead of maintaining a large if/else or switch statement, I use an `ISalaryCalculator` abstraction. New employee salary types can be introduced by creating another implementation without changing existing tested code. This follows OCP.

3. L — Liskov Substitution Principle (LSP)

Definition: A derived class should be usable anywhere its base class is expected **without breaking application behavior**.

Consider this design:

```
public abstract class Employee
{
    public abstract decimal CalculateBonus();
}
```

Permanent employee:

```
public class PermanentEmployee : Employee
{
    public override decimal CalculateBonus()
    {
        return 10000;
    }
}
```

Contract employees don't receive bonuses:

```
public class ContractEmployee : Employee
{
    public override decimal CalculateBonus()
    {
        throw new NotSupportedException(
            "Contract employees do not receive bonuses");
    }
}
```

```
}  
}
```

This is a design problem.

If we write:

```
public void ProcessBonus(Employee employee)  
{  
    var bonus = employee.CalculateBonus();  
}
```

then:

```
</> C#
```



works.

But:

```
</> C#
```



throws an exception.

The subtype cannot safely replace the parent type.

That violates LSP.

Better Design

Only employees eligible for bonuses should implement bonus behavior.

```
public abstract class Employee  
{  
    public string Name { get; set; }  
}
```

Create:

```
public interface IBonusEligible
{
    decimal CalculateBonus();
}
```

Permanent employee:

```
public class PermanentEmployee :
    Employee,
    IBonusEligible
{
    public decimal BasicSalary { get; set; }

    public decimal CalculateBonus()
    {
        return BasicSalary * 0.10m;
    }
}
```

Contract employee:

```
public class ContractEmployee : Employee
{
    public decimal HourlyRate { get; set; }
}
```

Now bonus processing accepts only eligible employees:

```
public void ProcessBonus(IBonusEligible employee)
{
    decimal bonus = employee.CalculateBonus();

    // Process bonus
}
```

Interview answer

A common LSP violation in payroll is putting `CalculateBonus()` in the base `Employee` class even though contract employees aren't eligible for bonuses. The contract employee then throws `NotSupportedException`. Instead, I introduce an `IBonusEligible` interface and implement it only for employee types that support bonus calculation.

4. I — Interface Segregation Principle (ISP)

Definition: Clients should not be forced to depend on methods they don't use.

Bad Design

Suppose we create:

</> C#



```
public interface IEmployee
{
    void CalculateSalary();

    void CalculateBonus();

    void CalculateOvertime();

    void CalculateCommission();
}
```

Now consider:

</> C#



```
public class ContractEmployee : IEmployee
{
    public void CalculateSalary()
    {
    }

    public void CalculateBonus()
    {
        throw new NotSupportedException();
    }

    public void CalculateOvertime()
    {
    }

    public void CalculateCommission()
    {
        throw new NotSupportedException();
    }
}
```

The contract employee is forced to implement functionality it doesn't need.

Better Design

Create smaller interfaces.

</> C#



```
public interface ISalaryEligible
{
    decimal CalculateSalary();
}
```

</> C#



```
public interface IBonusEligible
{
    decimal CalculateBonus();
}
```

</> C#



```
public interface IOvertimeEligible
{
    decimal CalculateOvertime();
}
```

</> C#



```
public interface ICommissionEligible
{
    decimal CalculateCommission();
}
```

Permanent employee:

</> C#



```
public class PermanentEmployee :
    ISalaryEligible,
    IBonusEligible,
    IOvertimeEligible
{
    public decimal CalculateSalary()
    {
        return 50000;
    }

    public decimal CalculateBonus()
    {
        return 5000;
    }
}
```

```
public decimal CalculateOvertime()
{
    return 2000;
}
}
```

Sales employee:

```
</> C#

public class SalesEmployee :
    ISalaryEligible,
    ICommissionEligible
{
    public decimal CalculateSalary()
    {
        return 40000;
    }

    public decimal CalculateCommission()
    {
        return 15000;
    }
}
```

Interview answer

In payroll, not every employee supports bonus, overtime, commission, PF, or gratuity. Instead of creating one large employee interface, I create focused interfaces such as `IBonusEligible`, `IOvertimeEligible`, and `ICommissionEligible`. Classes implement only the behavior they actually require.

5. D — Dependency Inversion Principle (DIP)

Definition: High-level modules should not depend directly on low-level modules. Both should depend on abstractions.

Consider payslip notification.

Bad Design

```
</> C#

public class PayrollProcessor
{
```

```

private EmailService _emailService;

public PayrollProcessor()
{
    _emailService = new EmailService();
}

public void ProcessPayroll()
{
    // Payroll processing

    _emailService.SendEmail();
}
}

```

PayrollProcessor is tightly coupled to EmailService .

Suppose tomorrow the requirement changes to:

- SMS
- Teams notification
- WhatsApp
- Push notification

We must modify PayrollProcessor .

Better Design

Create an abstraction.

```

</> C#

public interface INotificationService
{
    void Send(string message);
}

```

Email implementation:

```

</> C#

public class EmailNotificationService :
    INotificationService
{
    public void Send(string message)
    {
        Console.WriteLine(
            $"Email sent: {message}");
    }
}

```

...

```
}  
}
```

SMS implementation:

```
</> C#  
  
public class SmsNotificationService :  
    INotificationService  
{  
    public void Send(string message)  
    {  
        Console.WriteLine(  
            $"SMS sent: {message}");  
    }  
}
```

Payroll processor:

```
</> C#  
  
public class PayrollProcessor  
{  
    private readonly INotificationService  
        _notificationService;  
  
    public PayrollProcessor(  
        INotificationService notificationService)  
    {  
        _notificationService = notificationService;  
    }  
  
    public void ProcessPayroll()  
    {  
        // Salary calculation  
        // Tax calculation  
        // Payroll generation  
  
        _notificationService.Send(  
            "Your payroll has been processed.");  
    }  
}
```

ASP.NET Core Dependency Injection:

```
</> C#  
  
builder.Services.AddScoped<  
    INotificationService,  
    EmailNotificationService>();
```

```
builder.Services.AddScoped<PayrollProcessor>();
```

Now the dependency can easily be changed:

</> C#



```
builder.Services.AddScoped<
    INotificationService,
    SmsNotificationService>();
```

PayrollProcessor does not change.

Interview answer

In our payroll system, the payroll processing service should not directly create EmailService, repositories, or external services. I inject abstractions such as INotificationService and IPayrollRepository. ASP.NET Core's built-in DI container resolves the concrete implementations. This reduces coupling and makes unit testing much easier.

Putting All SOLID Principles Together

A more realistic service could look like this:

</> C#



```
public class PayrollProcessor
{
    private readonly ISalaryCalculator _salaryCalculator;
    private readonly IPayrollRepository _repository;
    private readonly INotificationService _notificationService;

    public PayrollProcessor(
        ISalaryCalculator salaryCalculator,
        IPayrollRepository repository,
        INotificationService notificationService)
    {
        _salaryCalculator = salaryCalculator;
        _repository = repository;
        _notificationService = notificationService;
    }

    public async Task ProcessAsync(Employee employee)
    {
        decimal salary =
            _salaryCalculator.CalculateSalary(employee);
```

```

        await _repository.SaveAsync(
            employee.Id,
            salary);

        await _notificationService.SendAsync(
            employee.Email,
            $"Payroll processed. Net Salary: {salary}");
    }
}

```

ASP.NET Core DI:

```

</> C#

builder.Services.AddScoped<
    ISalaryCalculator,
    SalaryCalculator>();

builder.Services.AddScoped<
    IPayrollRepository,
    PayrollRepository>();

builder.Services.AddScoped<
    INotificationService,
    EmailNotificationService>();

builder.Services.AddScoped<PayrollProcessor>();

```

Easy Way to Remember SOLID

Principle	Payroll example	Remember as
SRP	Separate calculation, DB, payslip and email	One job per class
OCP	Add new salary type without modifying existing calculators	Extend, don't modify
LSP	Contract employee shouldn't be forced to support bonus	Child must safely replace parent
ISP	Separate Bonus, Overtime and Commission interfaces	Small focused interfaces
DIP	PayrollProcessor depends on INotificationService	Depend on abstractions

A strong **experienced-level interview summary** is:

"In a payroll application, I use SRP to separate salary calculation, persistence, payslip generation and notifications. I use OCP so that new payroll rules or employee types can be introduced through new implementations rather than modifying existing logic. I follow LSP by ensuring derived employee types don't expose unsupported operations. I use ISP to keep payroll capabilities such as bonus, overtime and commission in focused interfaces. Finally, I use DIP and ASP.NET Core dependency injection so business services depend on abstractions such as repositories and notification interfaces rather than concrete implementations."

That answer is usually much stronger than simply giving textbook definitions because it connects **SOLID** → **C#** → **ASP.NET Core DI** → **real payroll business requirements**.



Sources



Convert to PDF

ChatGPT can make mistakes. Check important info.